

Лабораторная работа 2 **СОЗДАНИЕ ПРОСТЕЙШЕГО КЛИЕНТСКОГО ПРИЛОЖЕНИЯ В RAD STUDIO**

Цель работы: ознакомиться с разработкой клиентских приложений баз данных, позволяющих пользователям вводить/редактировать данные в таблицах БД; ознакомиться с принципами построения интерфейса на основе использования нескольких форм; разработать приложение для работы с базой данных;

ПОРЯДОК ВЫПОЛНЕНИЯ РАБОТЫ

1. Ознакомиться с теоретическими сведениями.
2. Создать приложение с контейнером DataModule
3. Организовать подключение к БД.
4. Обеспечить взаимодействие форм с наборами данных.
5. Создать главное меню приложения.
6. Ознакомиться с процессом изменения данных через DBGrid.
7. Реализовать программное редактирование данных в БД.
8. Ответить на контрольные вопросы.

ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

Для того чтобы ваше будущее приложение, написанное на любом языке программирования, могло подключиться к серверу **MySQL** и работать с данными необходимо установить **32-битную** версию драйвера ODBC (mysql-connector-odbc) соответствующую версии *MySQL* скачав его с официального сайта <https://dev.mysql.com/downloads/connector/odbc/>. В случае если будет вестись разработка **64-битного** приложения, то необходимо устанавливать две версии драйвера (**32-бит** и **64-бит**) поскольку среда разработки может иметь 32-битный интерфейс.

ODBC (*Open Database Connectivity*) – это программный интерфейс (*API*) доступа к базам данных, разработанный компанией *Microsoft*.

2.1 СОЗДАНИЕ ПРОСТЕЙШЕГО КЛИЕНТСКОГО ПРИЛОЖЕНИЯ

Для работы с нашей базой данных создадим новый проект. Для этого в меню **File** выберем пункт **New** и далее **Windows VCL Application – Delphi** (рис. 2.1). Появится окно с пустой формой, теперь необходимо сохранить проект в своей папке (например, **C:\User\Lab2**), снова выбрав меню **File** и нажав на **Save Project As**.

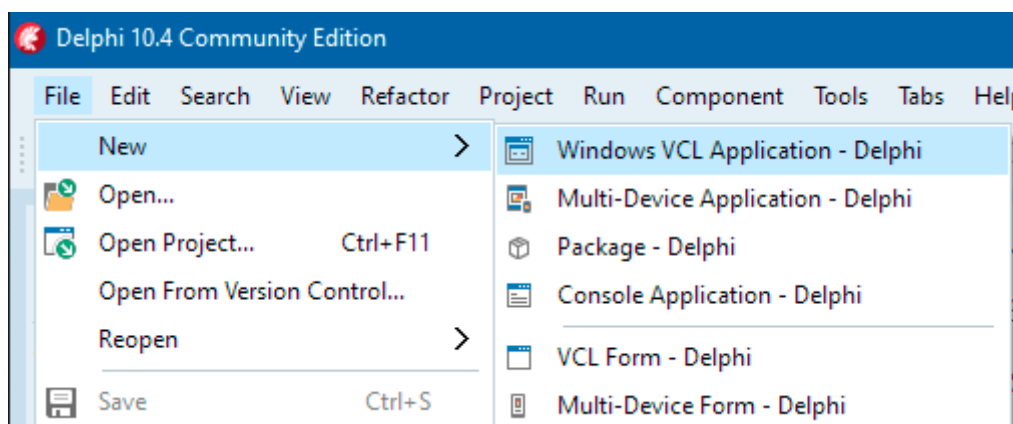


Рис. 2.1. Создание нового проекта на Delphi

Запустите проект на выполнение выбрав меню **Run** и нажав **Run (F9)**, должно построиться приложение с пустой формой.

При разработке программ, работающих с базой данных, в которых имеется более трех таблиц, для удобства пользователя используют интерфейс на основе нескольких форм. При этом возможна различная организация обращения к полям таблиц и их значениям, необходимых для реализации бизнес-правил.

Среда разработки **RAD Studio** предлагает размещать компоненты доступа к данным на специальной форме (модуле) – **DataModule**. Следует отметить, что контейнер **DataModule** не имеет ничего общего с классической формой приложения, поскольку на нем можно размещать только невидимые компоненты.

Добавим в проект модуль данных, для этого зайдите в главное меню **File** щелкните **New** и далее **Other**. В появившемся окне **New Items** (рис. 2.2) в группе **Database** выберем значок **Data Module**.

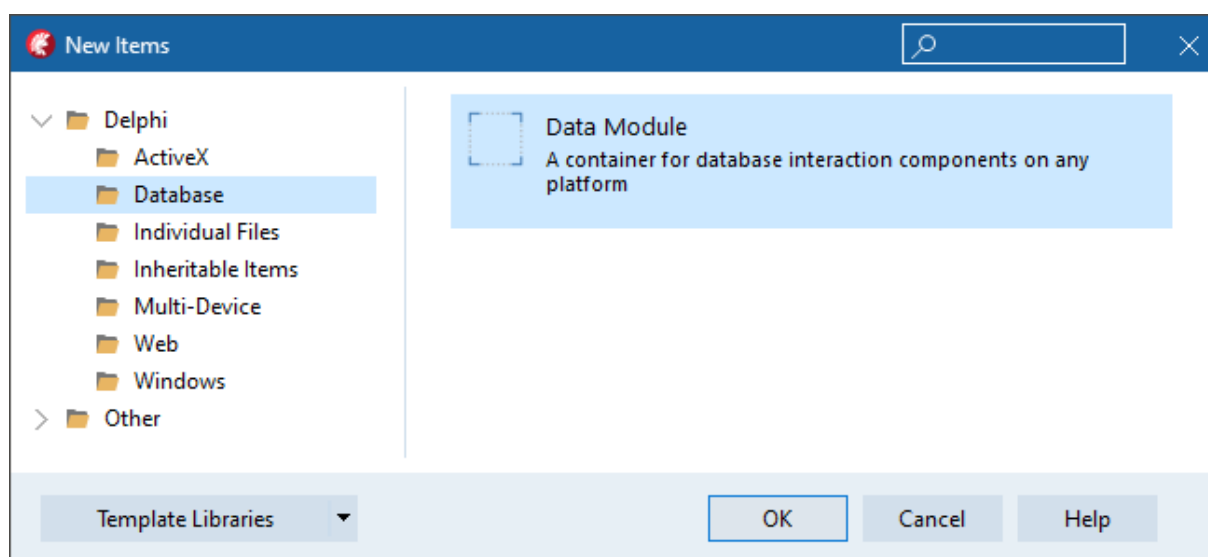


Рис. 2.2. Добавление к проекту компонента DataModule

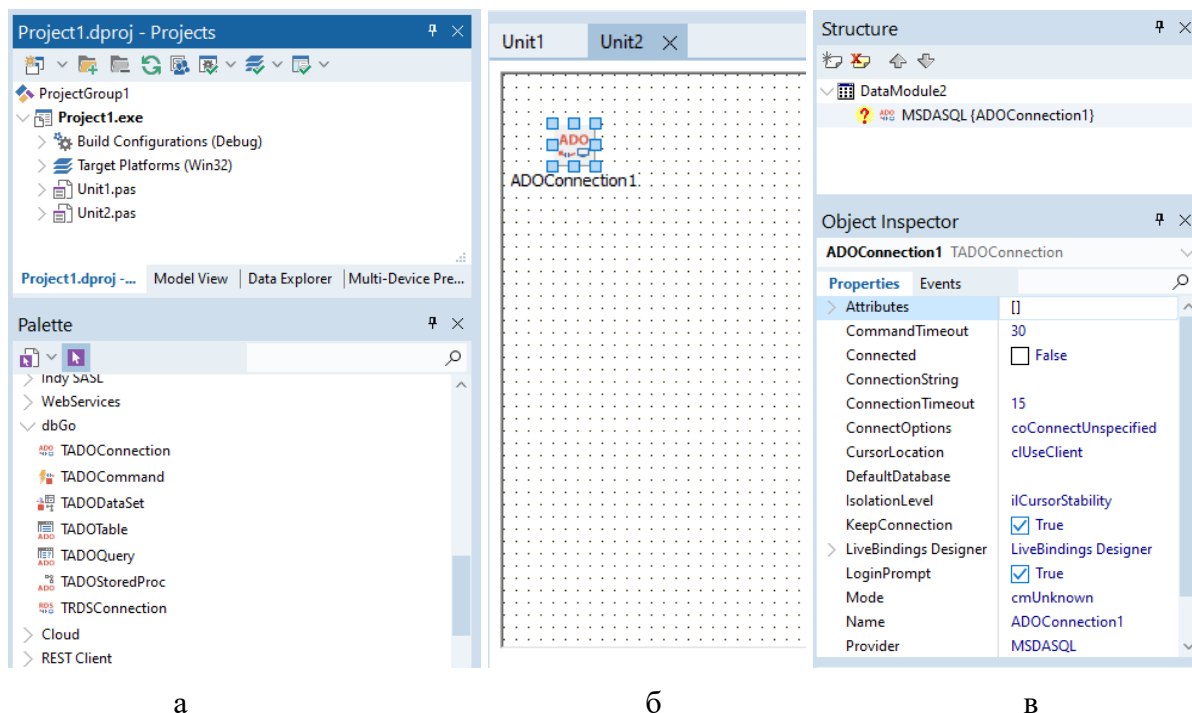
Контейнер **DataModule**, добавленный в приложение, автоматически приобретает название с цифрой следующего модуля (в нашем случае это **DataModule2**), который при необходимости можно переименовать.

Данный модуль доступен на этапе разработки и в то же время не является видимым для пользователя. Основным преимуществом размещения компонентов доступа к данным в контейнере **DataModule** является то, что изменение значения любого свойства компонента проявится сразу же во всем приложении.

Контейнер **DataModule** хранит только невизуальные компоненты, а весь интерфейс будет организован в виде отдельных форм.

2.2 ПОДКЛЮЧЕНИЕ К БАЗЕ ДАННЫХ СОЗДАННОЙ В MySQL

Для организации соединения с базой данных воспользуемся технологией **ADO**, используя невизуальный компонент **TADOConnection**, расположенный на вкладке **dbGO** палитры компонентов **Palette** (рис. 2.3, а). Выберем его мышью и разместим на форме (рис. 2.3, б). В инспекторе объектов **Object Inspector** на вкладке **Properties** (*Свойства*) будут отображены все основные свойства выбранного компонента (рис. 2.3, в).



а

б

в

Рис. 2.3. Фрагменты рабочих областей среды разработки:

а – менеджер проекта (**Projects**) и палитра компонентов (**Palette**);

б – фрагмент формы с установленным компонентом **TADOConnection**;

в – инспектор объектов, отображающий свойства компонента **ADOConnection1**

Для настройки соединения с базой данных выберем свойство **ConnectionString** компонента **ADOConnection1** и нажмем кнопку с многоточием. При этом появится окно формирования строки подключения (рис. 2.4), в котором нажмем на кнопку **Build**.

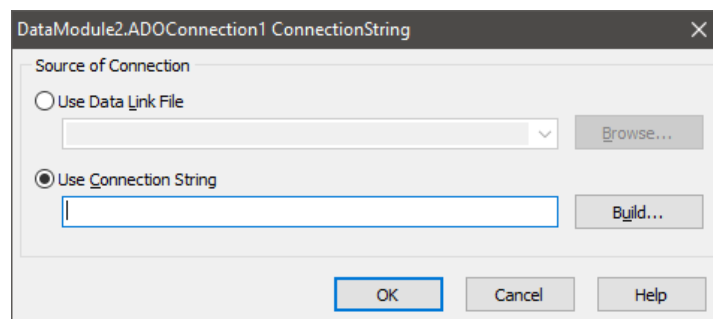


Рис. 2.4. Создание строки подключения в БД

Появится окно настройки источника соединения. В нем мы выбираем **Microsoft OLE DB Provider for ODBC Driver**, поскольку он позволяет подключаться к большинству современных СУБД (рис. 2.5). Сделав выбор, нажимаем на кнопку «Далее >>». На открывшейся вкладке **Соединение** настраиваем параметр **Использовать строку соединения** и нажимаем **Сборка**.

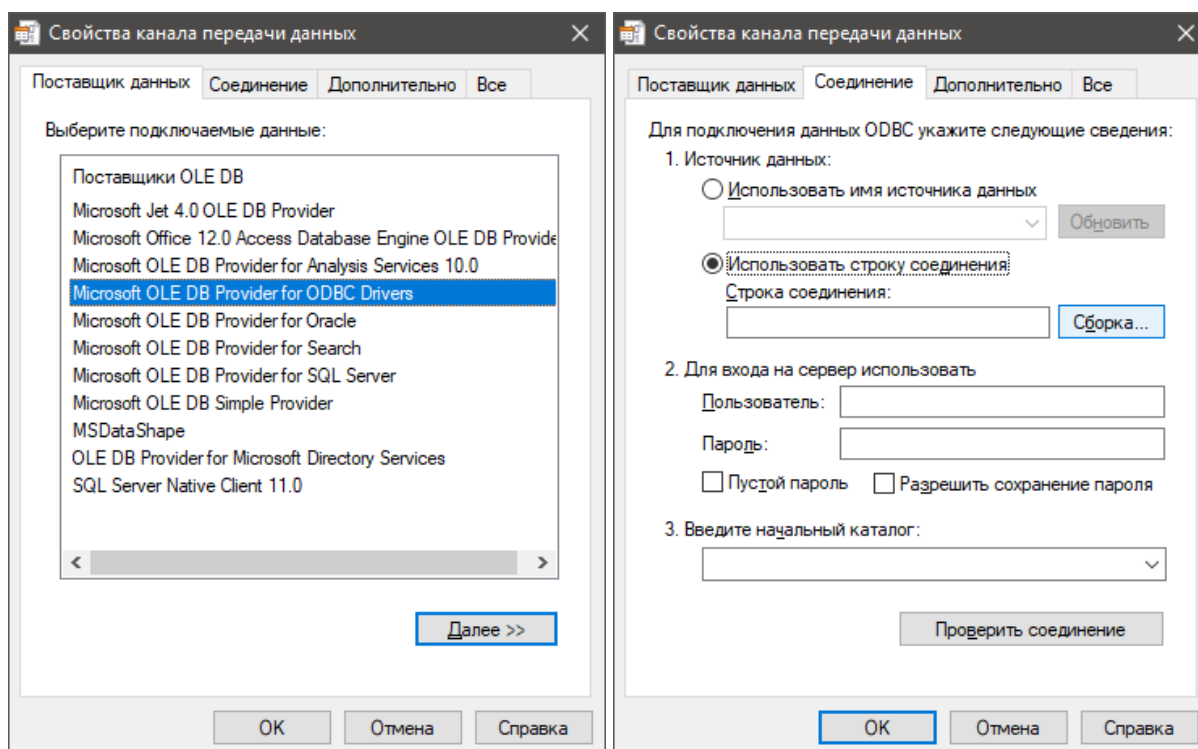


Рис. 2.5. Создание строки подключения в БД (продолжение)

В появившемся окне можно указать путь к файлу настроек подключения к базе данных (в случае если файл был ранее создан) или создать

новый. В нашем случае для начала необходимо создать новый файл с настройками подключения к БД. Для этого необходимо нажать на кнопку «Создать...» (рис. 2.6). После этого необходимо выбрать драйвер для соединения **MySQL ODBC Unicode Driver**.

В качестве примера показан выбор драйвера **MySQL ODBC** версии **5.3**, однако сейчас может быть установлена более свежая версия **8.***.

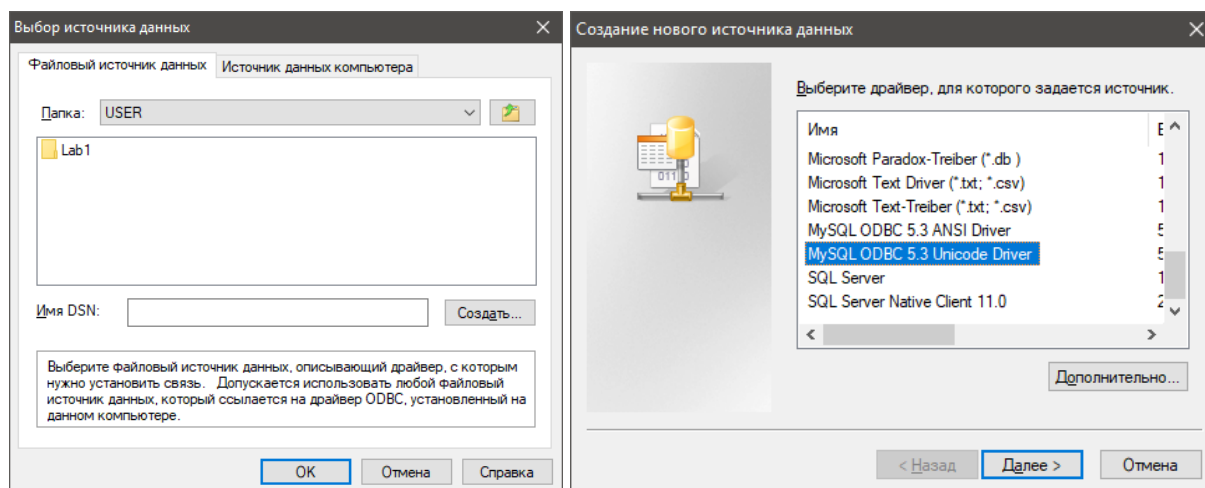


Рис. 2.6. Выбор драйвера подключения к БД

После выбора драйвера нажимаем на кнопку **Далее** и в новом окне нажимаем кнопку **Обзор**, указываем путь для сохранения файла настроек. После чего нажимаем на кнопку **Готово**. Появится окно настройки подключения к базе данных **MySQL** (рис. 2.7):

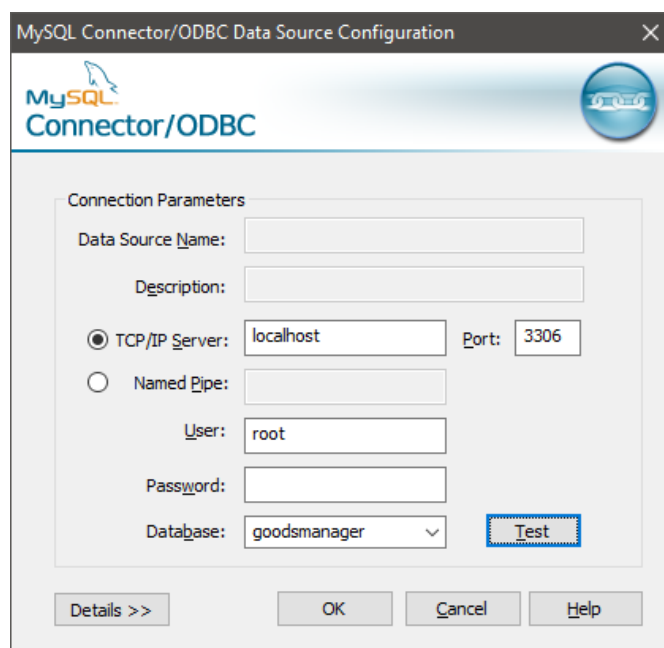


Рис. 2.7. Параметры подключения

Заполняем данные подключения и нажимаем **Test**. В случае успешного результата несколько раз подряд нажмем на кнопку **OK**.

Следует отметить, что поскольку база данных хранит данные в формате **utf8mb4** то для некоторых версий драйверов необходимо задать расширенные параметры, которые появляются при нажатии кнопки **Details**. Пример вида расширенных настроек для **MySQL ODBC** драйвера 8.0.27 приведен на рисунке 2.8.

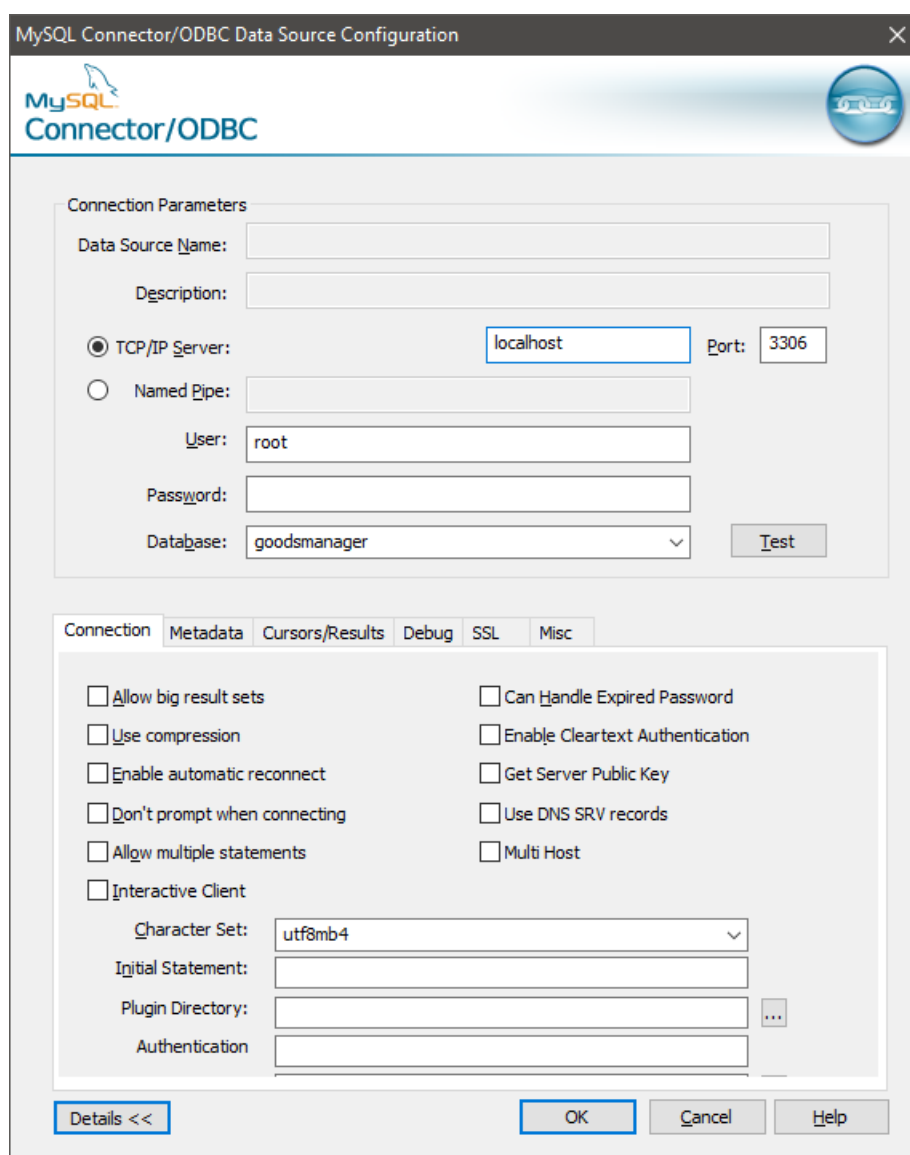


Рис. 2.8. Расширенные параметры подключения (MySQL ODBC Драйвер 8.0.27)

Далее для компонента **ADOConnection1** необходимо активировать свойство **Connected** в инспекторе объектов (*Object Inspector*). После настройки и активации соединения с базой данных можно приступить к добавлению компонентов, которые непосредственно будут использованы для работы с данными.

2.2.1 Компоненты для работы с данными

Завершив настройку соединения с базой данных, разместим на **DataModule2** шесть экземпляров невизуальных компонент **TADOTable** (один компонент на одну таблицу в базе данных), три компонента **TADOQuery** и девять компонент **TDataSource** (рис. 2.9).

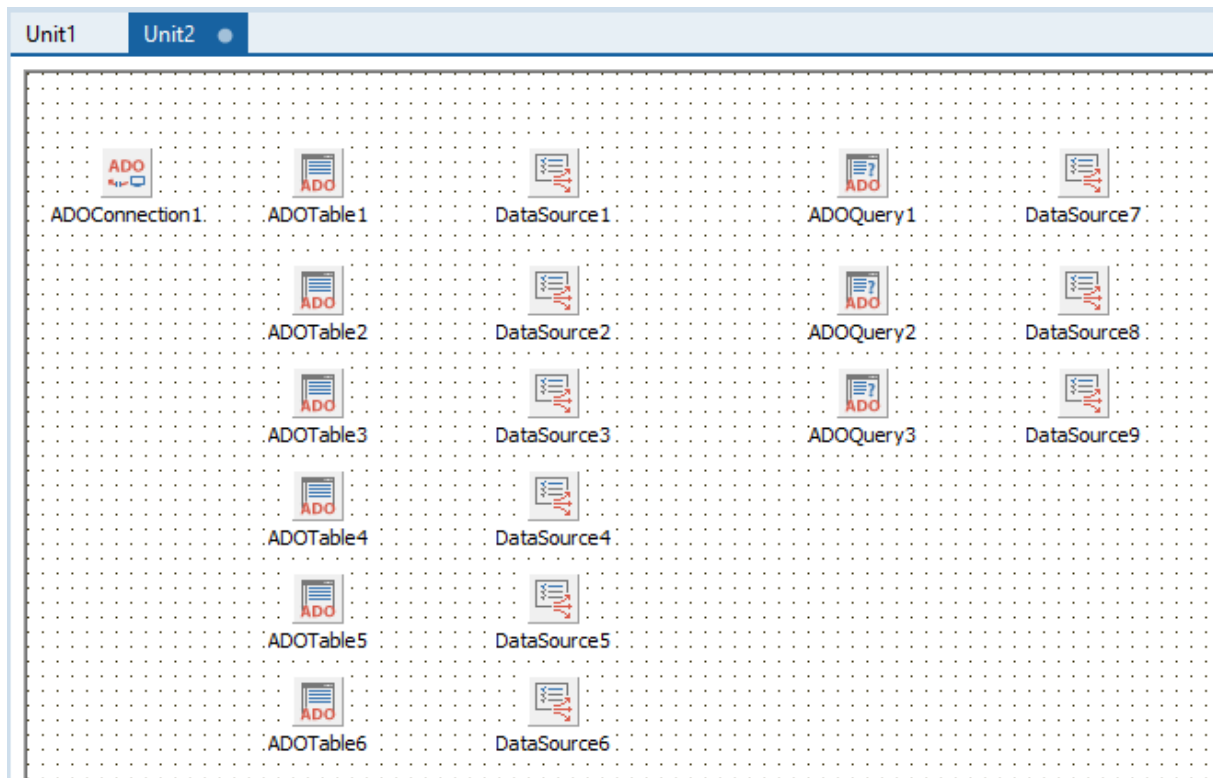


Рис. 2.9 – Пример размещения компонентов в **DataModule2**

Компоненты **TADOTable** и **TADOQuery** служат для хранения *наборов данных* (НД – данные таблиц, представлений, результатов запросов к базе данных).

Компонент **TDataSource** (*источник данных*), можно взять на странице палитры компонентов **Data Access**. Он служит связующим звеном (передает данные) между невизуальными компонентами (в данном случае компонентом **ADOTable** или **ADOQuery**) и визуальными компонентами, которые мы добавим на формы позднее.

Чтобы избежать путаницы в именах компонентов, дадим им осмысленные имена, как показано на рис 2.10. Для настройки имени компонента используется свойство **Name** в инспекторе объектов.

Поэтому введем следующие обозначения для имен компонентов: имена компонентов для работы с таблицами начинаются с префикса **T_**, для работы с запросами с **Q_**, с наборами данных с **DS_**.

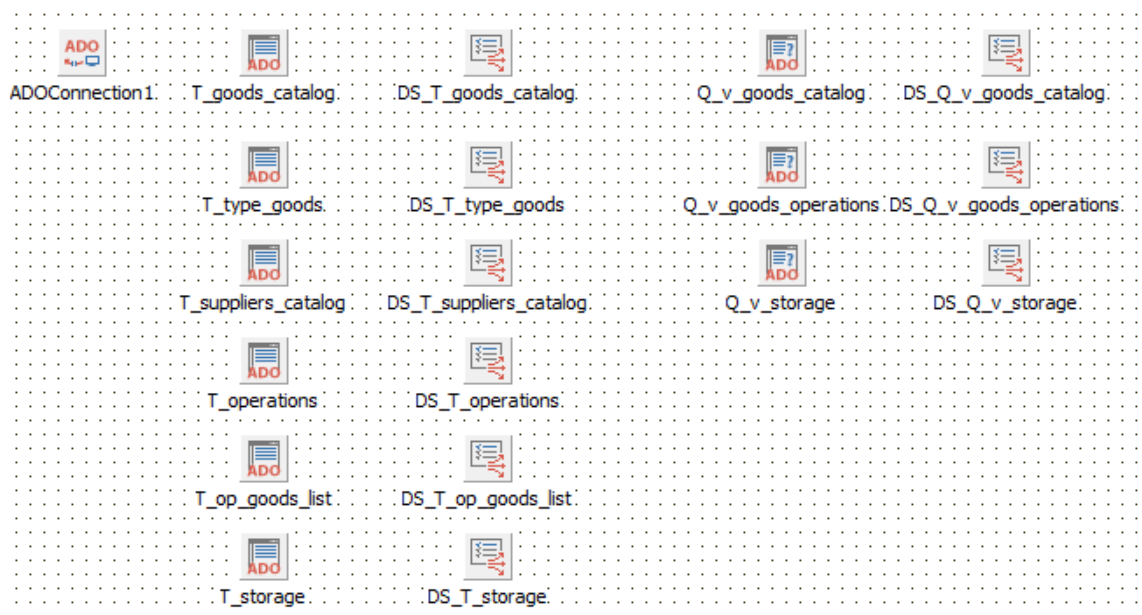


Рис. 2.10 – Размещения компонентов в DataModule2 с измененными именами

Теперь для *всех* компонентов **TADOTable** свойству **Connection** выставите **ADOConnection1**, далее назначьте свойства **TableName** именами *соответствующих* таблиц из базы данных, а затем активируйте свойства **Active** (рис 2.11):

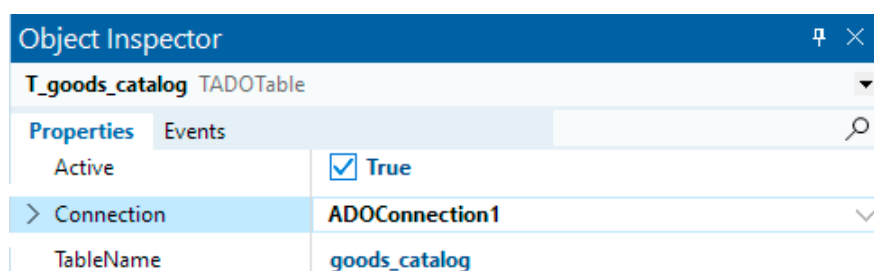


Рис. 2.11. Настройка свойств компонента ADOTable

Далее привяжем компоненты **TDataSource** к соответствующим компонентам **TADOTable**, установив свойства **DataSet** (рис 2.12).



Рис. 2.12. Настройка свойств компонента DataSource

Для организации работы с данными (отображение информации из нескольких таблиц и т.п.) в среде реляционных БД предпочтительным способом является использование языка **SQL** (*Structured Query Language* –

язык структурированных запросов), который поддерживается всеми современными реляционными СУБД. Для работы с запросами на языке *SQL* используются компоненты **TADOQuery**.

Рассмотрим пример настройки компонентов **TADOQuery** для выбора данных из ранее созданных представлений (рис 2.13). Основным свойством в данном случае является – **SQL**, оно настраивается первым. Однако не следует забывать, что для них также нужно настроить свойство **Connection**.

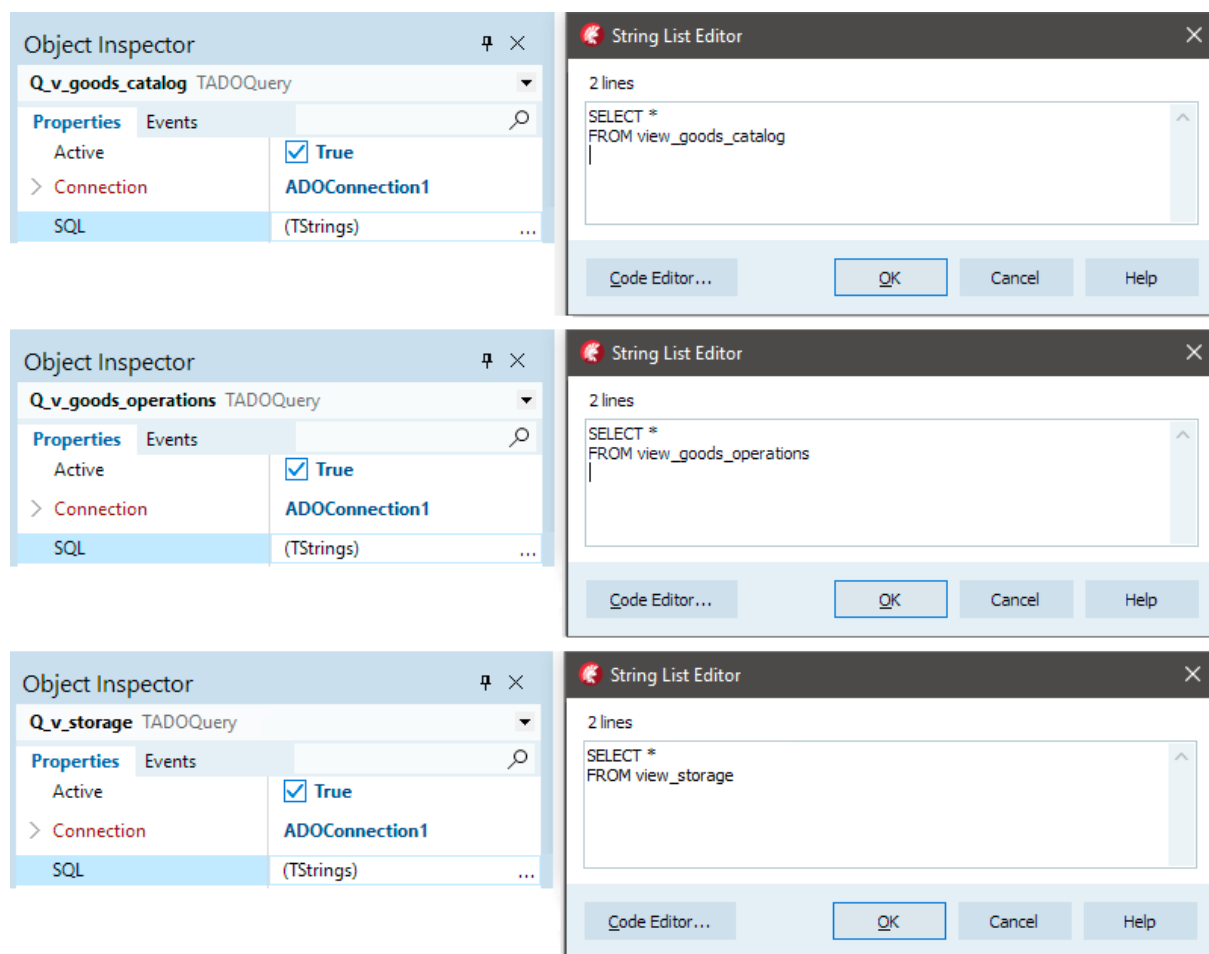


Рис. 2.13. Настройка свойств компонентов **ADOQuery**

Теперь необходимо связать оставшиеся компоненты **TDataSource** (*DS_Q_v_goods_catalog*, *DS_Q_v_goods_operations* и *DS_Q_v_storage*) с настроенными компонентами **TADOQuery**.

2.2.2 Настройка полей для набора данных и способы обращение к полям

При работе с наборами данных по умолчанию задействуются все поля из используемой таблицы базы данных (**TADOTable**) или же получен-

ные в результате выполнения запроса (**TADOQuery**). Этот способ всегда действует по умолчанию. Для обращения к полям таблиц с использованием имен в RAD Studio необходимо задать поля через Редактор полей (**Field Editor**). Он позволяет включить в состав отображаемых полей все поля или необходимое подмножество полей. Для вызова редактора полей необходимо выделить компонент (**TADOTable** или **TADOQuery**) и в контекстном меню выбрать элемент **Fields Editor**.

Рассмотрим настройку на примере компонента **T_suppliers_catalog**. В списке редактора полей, пока он пуст, нажмем на правую кнопку мыши и во всплывающем меню выберем элемент меню **Add fields**. При этом будет показан список всех полей таблицы **suppliers_catalog**. Если мы хотим использовать только часть полей, то отметим при помощи мыши и клавиши **Shift** необходимые поля. Теперь список редактора полей будет включать все отмеченные поля. Если нужно добавить все поля, то можно воспользоваться опцией контекстного меню – **Add all fields** (рис 2.14).

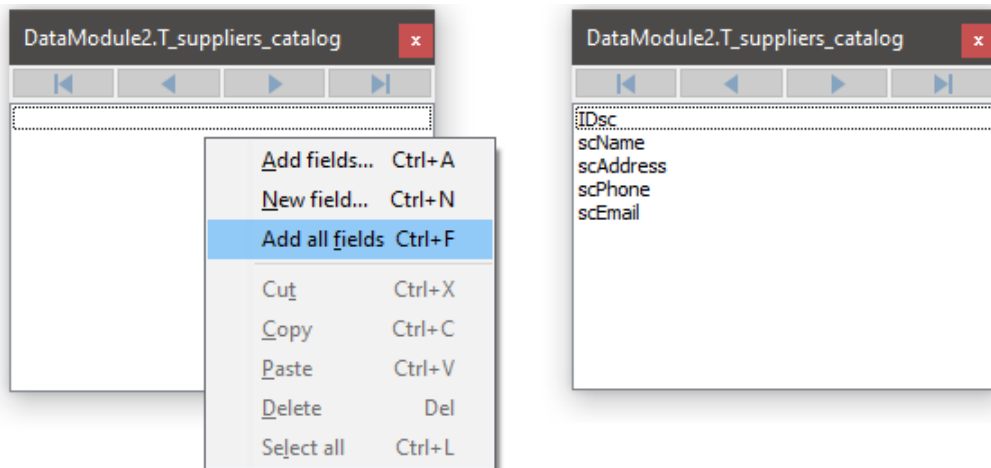


Рис. 2.14. Пример настройки состава полей у компонента **T_suppliers_catalog**

В составе столбцов компонента **TDBGrid**, который связан с данным набором данных будут присутствовать только те поля, которые определены в редакторе полей.

К полям и можно обратиться, несколькими способами :

- через имя данного компонента, определяемого свойством **Name**, если полю соответствует компонент **TField**, например:

```
T_suppliers_catalogscName.Value := 'Моя компания';
```

- используя метод **FieldByName** набора данных:

```
T_suppliers_catalog.FieldByName('scName').Value := 'Моя компания';
```

- используя свойство **Fields** набора данных, задавая индекс:

```
T_suppliers_catalog.Fields[1].Value := 'Моя компания';
```

Индекс является порядковым номером поля в определении таблицы базы данных. Отсчет идет от нуля. Например, если поле **scName** определено в таблице базы данных вторым по счету, то его индекс будет равен 1:

– используя свойство набора данных **FieldValues**, которое позволяет обращаться к полю через его имя, указываемое как содержимое параметра **FieldName**, например:

```
T_suppliers_catalog.FieldValues['scName'] := 'Моя компания';
```

Это свойство принимается для набора данных по умолчанию, поэтому его имя при обращении к полю можно опускать, например:

```
T_suppliers_catalog['scName'] := 'Моя компания';
```

Более предпочтительным считается обращение к полю через его имя или через метод **FieldByName**, поскольку в этом случае мы обращаемся к конкретному полю по его имени, исключая возможность обращения к несуществующему полю.

Менее предпочтительным является обращение к полю через свойство набора данных **Fields**. Это связано с тем, что если поле **scName** было объявлено в таблице базы данных вторым по счету (например, обращение **Fields[1]**), а затем удалено из структуры этой таблицы, то это обращение в программном коде будет воспринято как обращение ко второму полю в ТБД. Однако фактически этим полем будет третье поле, следовавшее за полем **scName** перед тем, как оно было удалено. Таким образом, возникает алгоритмическая ошибка, которая может быть не распознана, если поле **scName** и поле, ставшее после удаления поля **scName** вторым, имеют одинаковые или совместимые типы. Такие ошибки очень трудно локализовать. Поэтому следует по возможности воздерживаться от обращения к полю через свойство **Fields**.

2.3 ОРГАНИЗАЦИЯ ВЗАИМОДЕЙСТВИЯ ФОРМ С НАБОРАМИ ДАННЫХ В КОНТЕЙНЕРЕ **DATA MODULE**

При создании приложения будем использовать три формы: на первой форме будет отображаться информация по операциям с товаром, на второй – сведения о поставщиках, на третьей – информация о товаре и типах товара. Поскольку первая форма была сформирована в момент создания проекта, то необходимо добавить еще две формы. Для этого в меню **File** у **Delphi (RAD Studio)** необходимо выбрать **New** и щелкнуть **VCL Form – Delphi**. После добавления двух форм в проекте должно быть 4 файла с исходным кодом (*Unit*.pas – pascal*) привязанных каждый к своей форме (*Unit*.dfm – Delphi form*) (рис 2.15).

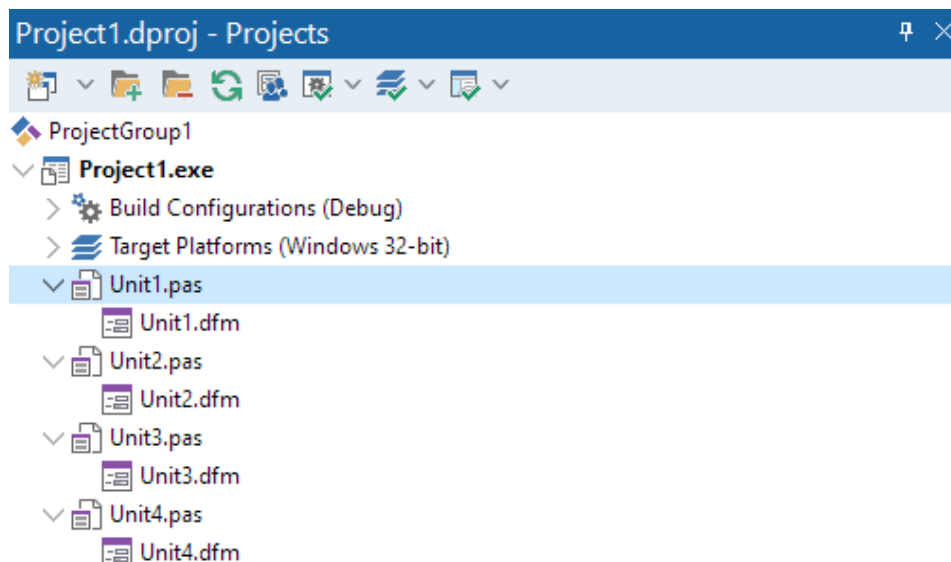


Рис. 2.15 – Состав проекта на Delphi

На визуальных формах будем размещать управляющие компоненты: кнопки, поля ввода, меню и т.д. По умолчанию главной считается первая созданная форма, она изначально появляется на экране при запуске проекта (главную форму можно поменять в настройках проекта).

Формы в проекте пока не связаны между собой, поэтому мы не можем обращаться из программного кода одной формы к компонентам других форм. Для организации взаимосвязи между формами и/или модулями, для главной формы выберите пункт меню **File** и нажмите **Use Unit...**, после чего появится окно связи модулей проекта (рис. 2.16). В связи с тем, что первая форма будет вызывать все остальные, выберем все модули и нажмем кнопку **ОК**.

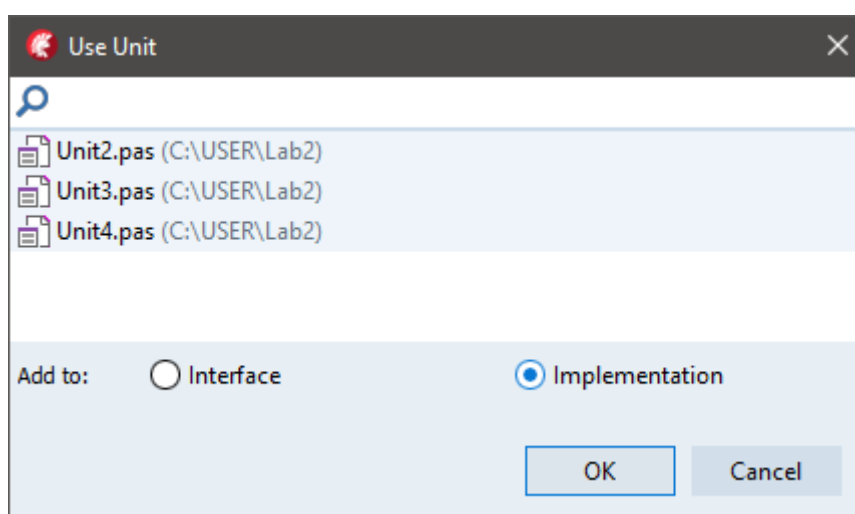


Рис. 2.16 – Создание связи между модулями проекта

На **Form1** будут отображаться сведения о состоянии склада и операциях с товаром. Для этого разместим компонент **TPageControl** который позволяет организовывать просмотр контента в виде вкладок. Чтобы доступная область данного компонента покрывала всю форму, необходимо свойству **Align** задать значение **alClient** (рис 2.17). Для добавления вкладки необходимо вызвать контекстное меню и выбрать пункт **New Page**.

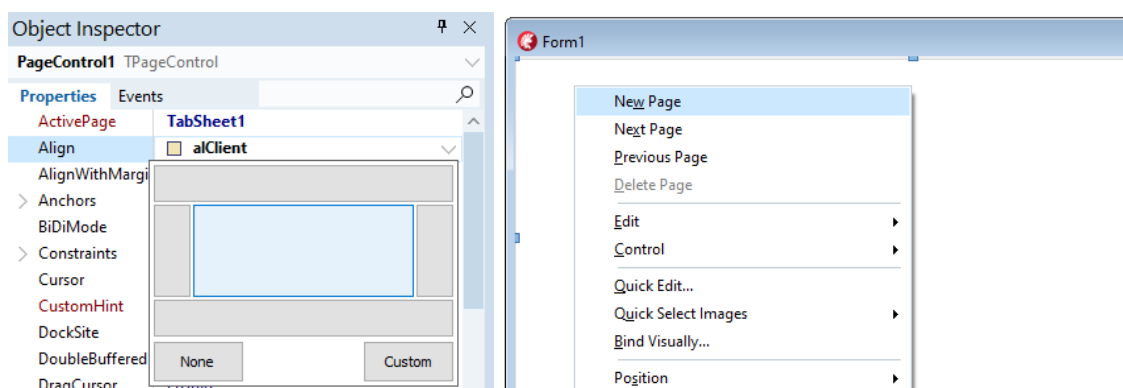


Рис. 2.17 – Настройка свойства **Align** и контекстное меню для создания вкладки.

Поскольку на форме будут показываться складские запасы и сведения об операциях, добавим две вкладки (рис 2.18).



Рис. 2.18 – **PageControl** с двумя созданными вкладками.

Изменим свойства **Caption** у вкладок **TabSheet** чтобы было понятно их назначение. После этого поместим на обеих вкладках **TabSheet** по компоненту **TDBGrid**. Далее присвойте свойству **DataSource** для

- **DBGrid1** значение **DataModule2.DS_Q_v_storage**
- **DBGrid2** значение **DataModule2.DS_Q_v_goods_operations**.

У компонента **DBGrid1** выставите свойству **Align** (Выравнивание) – значение **alClient** (рис 2.19):

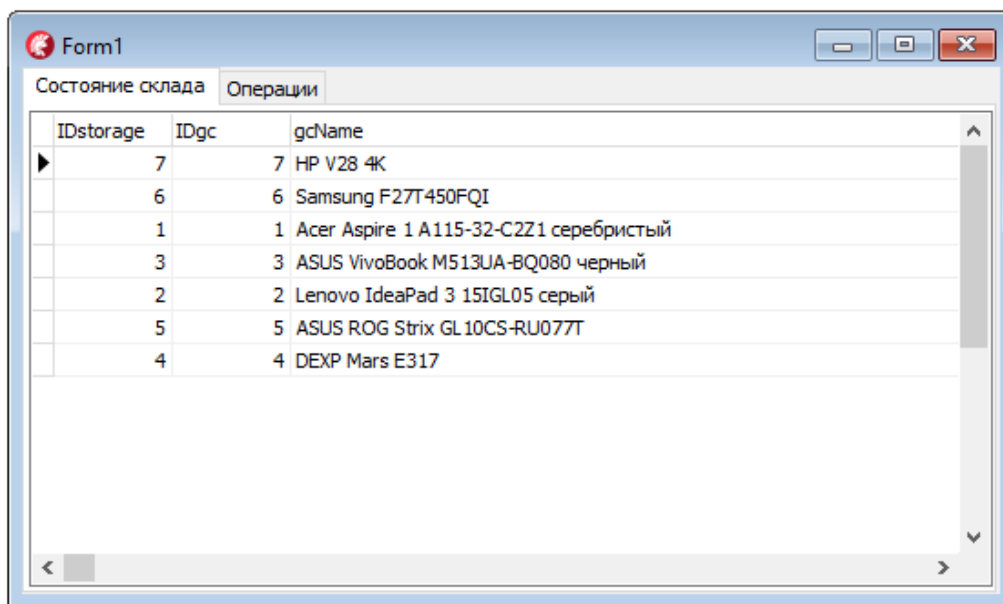


Рис. 2.19 – Отображение данных НД `Q_v_storage` в `DBGrid1`

Для того чтобы организовать в программе переход между формами добавим в программу меню.

2.4 СОЗДАНИЕ ГЛАВНОГО МЕНЮ И ВЫЗОВ ФОРМ

Чтобы добавить к программе главное меню, нужно разместить на форме в произвольном месте невидимый компонент `TMainMenu`. Опции главного меню создаются с помощью специального редактора. Редактор меню вызывается с помощью двойного щелчка по компоненту или при помощи пункта контекстного меню **Menu Designer**. Первоначально меню пустое, но имеет один выделенный элемент (рис. 2.20).

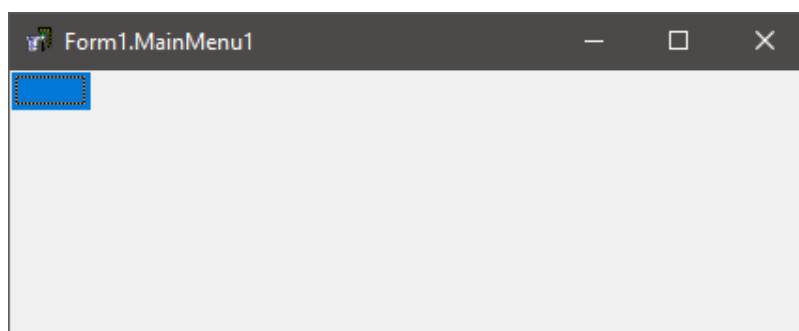


Рис. 2.20. Окно дизайнера меню

Для создания первой опции (как правило, это опция главного меню **Файл**) нужно перейти в инспектор объектов и свойству **Caption** присвоить нужное название.

Обратите внимание на то, что среда **Delphi** автоматически создает следующий пустой пункт меню верхнего уровня. При выборе одного из

пунктов этого меню (например, **Файл**) мы получим пустой пункт меню второго уровня. Выделим нижний элемент меню, в инспекторе объектов изменим свойство **Caption** на **Выход**.

В контекстном меню элементов второго уровня имеется пункт **Create SubMenu**, нажав на который, можно создать подменю выбранного элемента, а к его названию прибавится изображение треугольника – стрелки, указывающей на его наличие. Работа с подменю осуществляется так же, как и с элементами основного меню.

Создадим в главной строке меню два пункта **Поставщики** и **Управление товарами**. Теперь настроим действия пунктов меню.

Сначала выберите пункт меню **Выход**, перейдите на вкладку инспектора объектов – **Events** (*События*) и щелкните дважды мышью на пустую строчку на против события **OnClick** (рис 2.21):

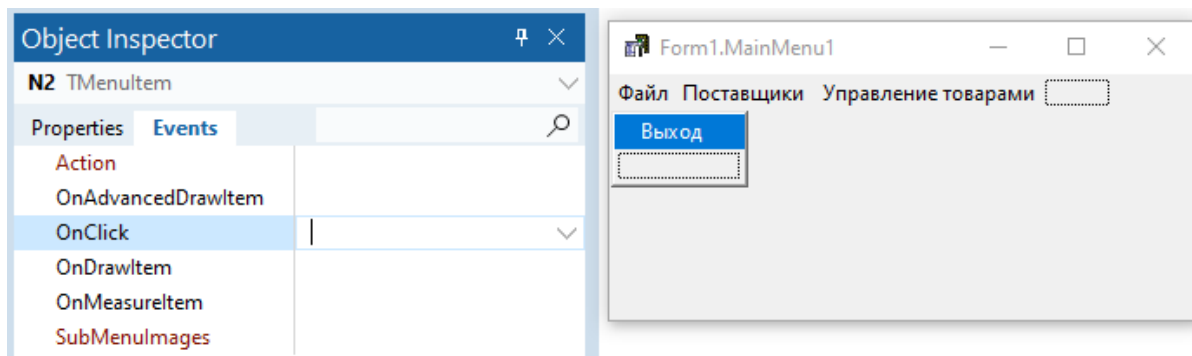


Рис. 2.21. Назначение обработчика события «Выход»

и запишем следующий текст в обработчике:

```
Close;
```

или

```
Application.Terminate;
```

Данный код будет закрывать приложение при клике мышью на пункт меню **Выход**.

Привяжем к пунктам меню вызов остальных форм приложения. Для пункта **Поставщики** в обработчик **OnClick** необходимо вписать код одного из двух способов показа нужной формы:

– в случае обычной формы:

```
Form3.Show;
```

– в случае модальной формы:

```
Form3.ShowDialog;
```

Разница между этими формами заключается в том, что обычная форма разрешает свободный переход между всеми формами, находящимися в данный момент на экране, а модальная форма в момент вызова блоки-

рует переход между формами проекта до тех пор, пока не будет закрыта, и работа возможна только в ней.

Для пункта **Управление товарами** в обработчик **OnClick** добавьте вызов четвертой формы.

Теперь необходимо задать связь между формой **Form3**, отвечающей за поставщиков, и модулем **DataModule (Unit2)**, используйте для этого команду **Use Unit**. После этого разместим на форме компонент **TDBGrid** и свяжем его с набором данных, содержащим таблицу **Поставщики (suppliers_catalog)** (рис 2.22).

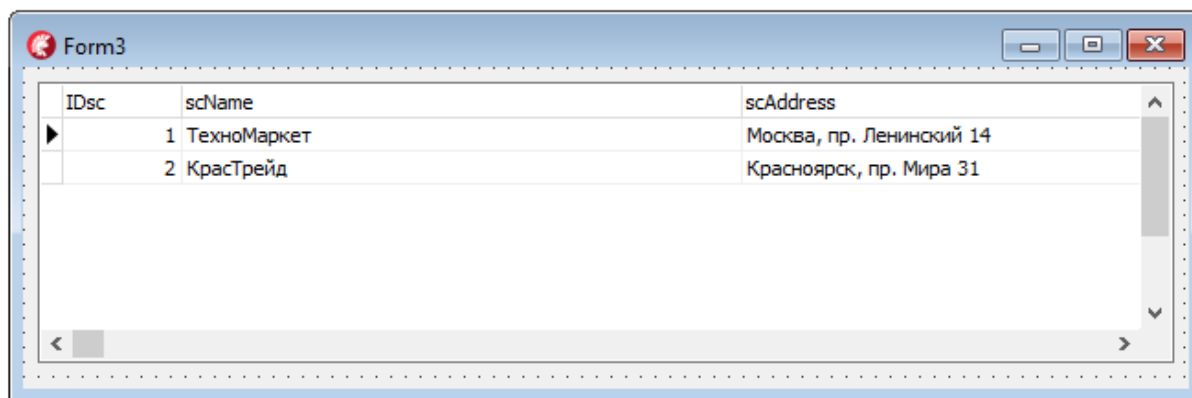


Рис. 2.22 – Отображение данных таблицы **Поставщики**

Сохраним проект, используя элемент меню **Save All** или одноименную пиктограмму на панели инструментов, и откомпилируем приложение с помощью элемента меню **Run→Run** или клавиши **F9**.

Добавлять записи в набор данных и, соответственно, в таблицу **suppliers_catalog** можно непосредственно из компонента **TDBGrid**. Для этого нажмем на клавиатуре клавишу **Insert** или, находясь на последней записи набора данных, клавишу управления курсором **↓** (рис 2.23). Набор данных автоматически перейдет в режим добавления новой записи. После ввода значений в поля записи можно запомнить запись в наборе данных, перейдя на другую запись при помощи клавиш управления курсором. Отказаться от запоминания записи можно, нажав клавишу **Esc**. Следует отметить, что в поля автоинкрементного типа нельзя вводить данные, тем не менее при сохранении записи им будет автоматически присвоено новое значение.

Для изменения записи переместим указатель текущей записи в нужное место и изменим значения там, где это необходимо. Набор данных автоматически перейдет в режим редактирования. Для удаления записи следует установить на нее указатель текущей записи и нажать комбинацию клавиш **Ctrl + Del**.

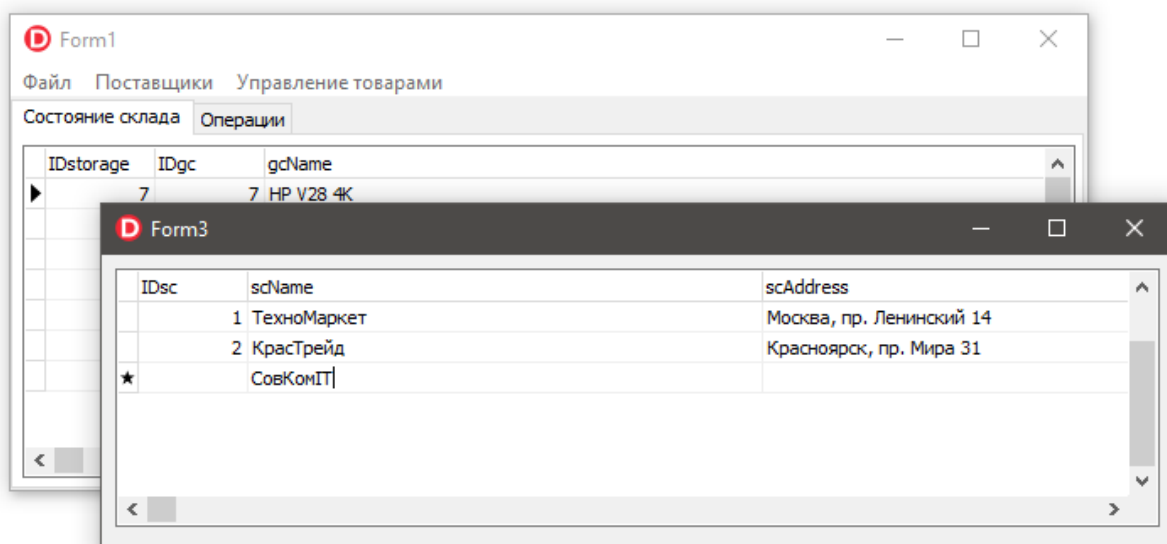


Рис. 2.23 – Редактирование данных через **DBGrid**

Можно запретить добавление и изменение данных через компонент **TDBGrid**, установив для таблицы **T_suppliers_catalog** свойство **ReadOnly** в значение **True**. В таком случае придется воспользоваться подходом редактирования, который будет рассмотрен ниже.

2.5. РЕДАКТИРОВАНИЕ ДАННЫХ И СИНХРОНИЗАЦИЯ ОТОБРАЖЕНИЯ РЕЗУЛЬТАТОВ ЗАПРОСА

На **Form4** для организации вывода каталога товаров типов товаров также воспользуемся компонентом **TPageControl**. Зададим для его свойства **Align** значение **alClient**. Добавим две вкладки, на которые поместим по компоненту **TDBGrid**. У них также выставим свойству **Align** (Выравнивание) – значение **alClient**. Далее свойству **DataSource** присвоим следующие значения (рис 2.24):

- **DBGrid1** значение **DataModule2.DS_Q_v_goods_catalog**.
- **DBGrid2** значение **DataModule2.DS_T_type_goods**.

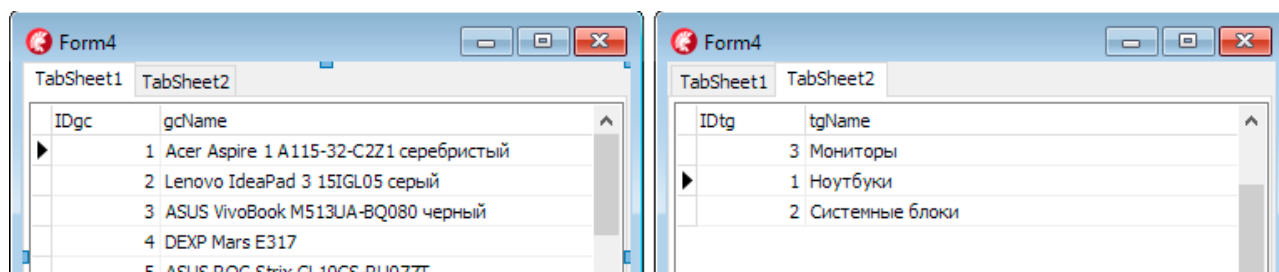


Рис. 2.24 – **TabSheet** с компонентами **TDBGrid**.

Поскольку нам не требуется отображать пользователю все столбцы из набора данных каталога товаров (**Q_v_goods_catalog**) то мы настроим

их отображение при помощи редактора столбцов компонента **TDBGrid**. Для этого щелкнем правой кнопкой мыши на компоненте **DBGrid1** и в контекстном меню выберем элемент **Columns Editor**. На экране появится окно редактора столбцов компонента. Далее щелкнем на кнопке **Add All Fields**, в результате чего будут добавлены столбцы, каждый из которых соответствует полю **Q_v_goods_catalog** (рис. 2.25).

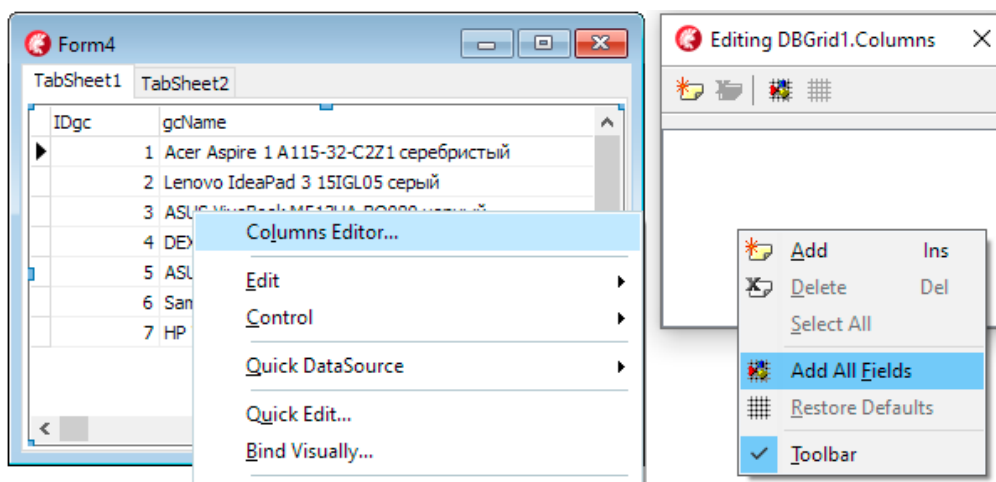


Рис. 2.25 – Добавление полей в **TDBGrid**.

Для отображения в таблице нам будет достаточно полей, показывающих **тип товара**, его **название** и **цену**, поэтому оставим только их. Чтобы изменить заголовок каждого столбца, выберем при помощи мыши имя столбца в редакторе столбцов и раскроем список свойства **Title** в инспекторе объектов. Свойство **Caption** определяет заголовок столбца. Для изменения ширины столбца настроим параметр **Width**. Изменим соответствующим образом заголовки и при необходимости – параметр ширины столбцов (рис. 2.26).

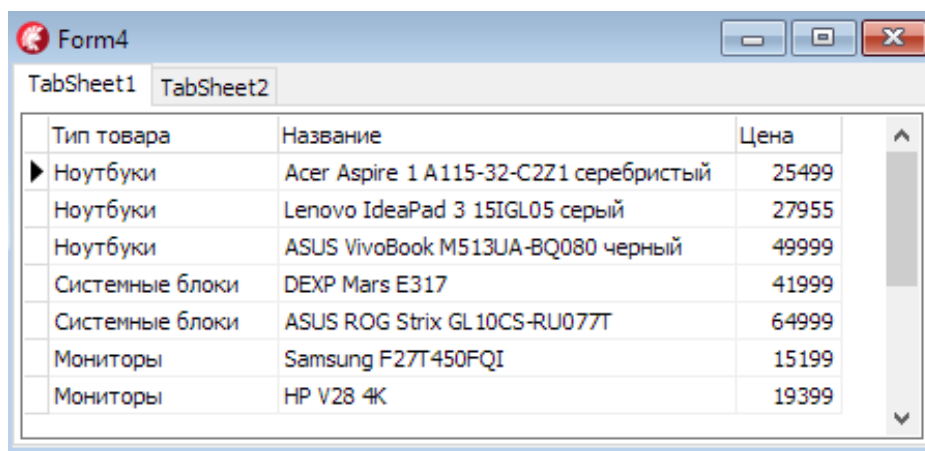


Рис. 2.26 – Изменение отображения столбцов в **DBGrid1**.

Аналогичным образом настроим заголовки у **DBGrid2** на **TabSheet2**, который отображает данные таблицы **type_goods**.

Вывод описания товара будет происходить только для выделенной записи. Для этого нам необходимо на **TabSheet1** разместить два компонента: **TLabel** (Метка) и **TDBText** (Текстовое поле из БД). Назначьте свойства как указано в таблице 2.1.

Таблица 2.1

Свойства компонентов для вывода описания товаров

Свойство	Значение
Label1	
Align	alBottom
Caption	Описание:
DBText1	
Align	alBottom
DataSource	DataModule2.DS_T_goods_catalog
DataField	gcDescription
AlignWithMargins	True
AutoSize	True
WordWrap	True

Пример отображения компонентов после выполнения настройки приведен на рис. 2.27

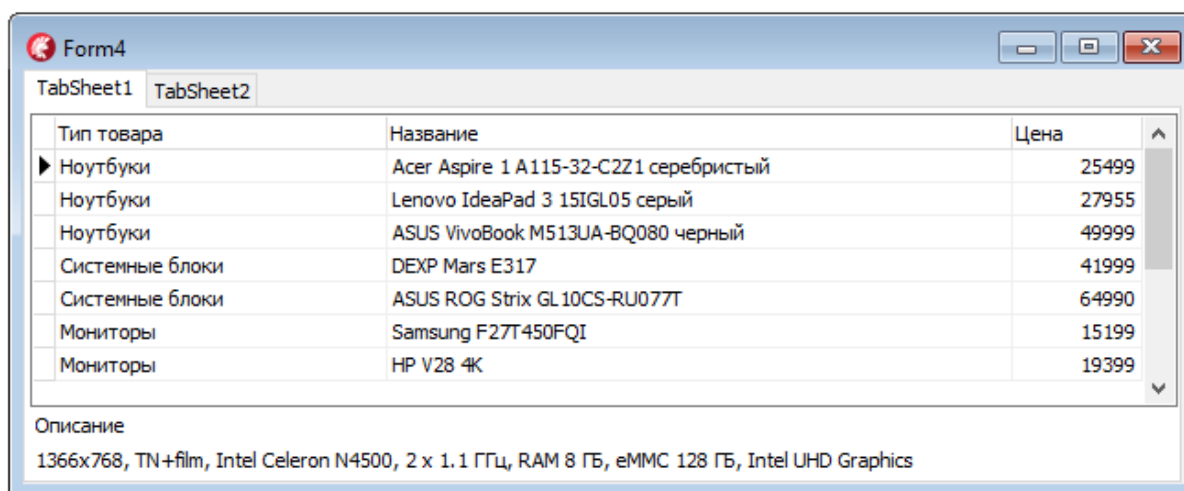


Рис. 2.27 – Вид каталога товаров после настройки компонентов **Label1** и **DBText1**.

Для того чтобы сделать автоматическое размещения одного компонента под другим используется свойство **Align** (Выравнивание) с значением **alBottom** (Снизу). При стандартном значении текст будет стоять достаточно плотно к окружающим компонентам, и для того чтобы избежать этого нужно свойству **AlignWithMargins** (Выравнивание с отступами) назначить значение **True**, сами же параметры отступов можно отрегулировать раскрыв блок **Margins** (Отступы) и при необходимости скорректиро-

вать значения. Автоматическая подстройка размеров компонента задается при помощи свойства **AutoSize** (Авторазмер). В случае длинного текстового пояснения необходимо организовать вывод в несколько строк, для этого свойству **WordWrap** (Перенос слов) задается значение **True**.

Далее поместим на первой вкладке **TabSheet1** два компонента **TPanel**. Назначим им свойства согласно таблице 2.2.

Таблица 2.2

Свойства компонентов для вывода описания товаров

Свойство	Значение
Panel1	
Align	alBottom
Name	InsertEditDeletePanel
Panel2	
Align	alBottom
Name	PanelToInputValues
Visible	False

На **Panel1** (InsertEditDeletePanel) установим три кнопки (компонент **TButton**). Для задания отображаемой надписи у большинства компонентов существует свойство **Caption**, для обращения к их свойствам из программного кода используется свойство **Name**. Зададим свойства **Caption/Name** для кнопок следующим образом **Вставить/InsertButton**, **Изменить/EditButton**, **Удалить/DeleteButton** (рис. 2.28).

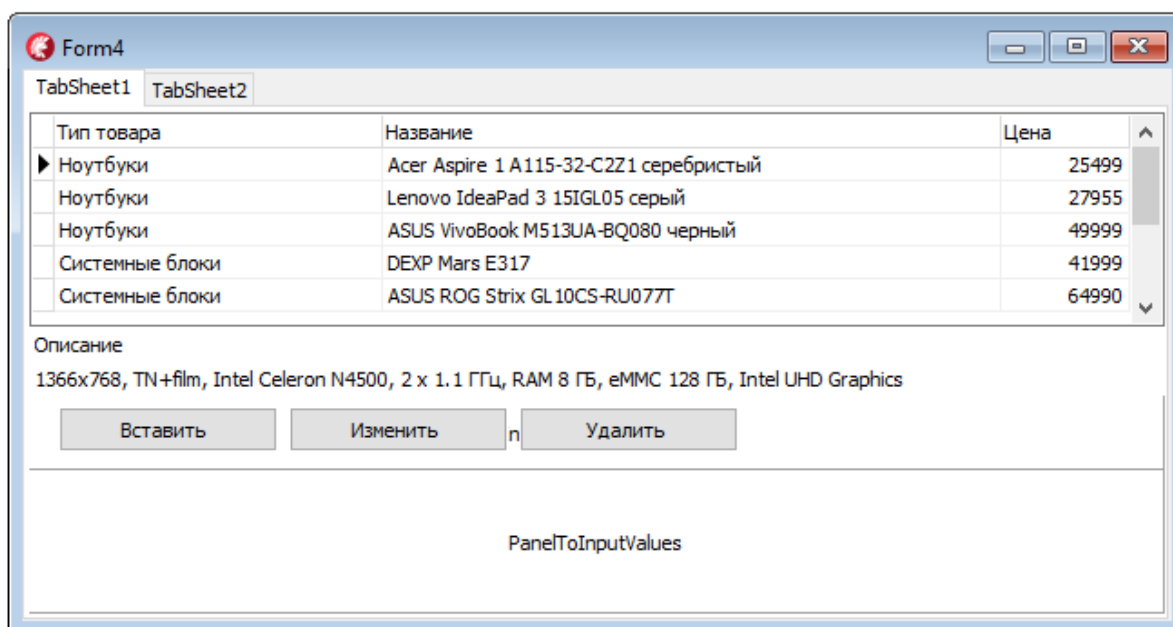


Рис. 2.28. Результат добавления компонентов на **Panel1**

На **Panel2** (PanelToInputValues) расположим один компонент **TDBLookupComboBox** и три компонента, **TDBEdit**, со страницы палитры компонентов **Data Controls**. Данные компоненты позволят изменять зна-

чения соответствующих полей. Для пояснения компонентов редактирования поставим компоненты **TLabel**. Для сохранения или отказа от сделанных изменений в таблице поместим на панели кнопки **Запомнить/PostButton** и **Отменить/CancelButton** (рис. 2.29).

The screenshot shows a Windows application window titled "Form4". It contains a tabbed interface with "TabSheet1" and "TabSheet2". On "TabSheet1", there is a table with three columns: "Тип товара" (Product Type), "Название" (Name), and "Цена" (Price). The table has four rows of data. Below the table is a text area labeled "Описание" (Description) containing technical specifications. Under the description are three buttons: "Вставить" (Insert), "Изменить" (Edit), and "Удалить" (Delete). At the bottom of the form, there are three input fields: "Label2" (a dropdown menu), "Label3" (a text box), and "Label4" (a text box). Below these is a "Запомнить" (Remember) button and an "Отменить" (Cancel) button.

Рис. 2.29 – Размещение компонентов на **PanelToInputValues**

Теперь настроим компоненты, чтобы пользовательский интерфейс формы соответствовал рис. 2.30, при этом свойство **Caption** у панелей сделаем пустым.

Таблица 2.2

Свойства компонентов для редактирования таблицы **goods_catalog**

Свойство	Значение
DBLookupComboBox1	
DataSource	DataModule2.DS_T_goods_catalog
DataField	IDtg
ListSource	DataModule2.DS_T_type_goods
ListField	tgName
KeyField	IDtg
DBEdit1	
DataSource	DataModule2.DS_T_goods_catalog
DataField	gcName
DBEdit2	
DataSource	DataModule2.DS_T_goods_catalog
DataField	gcCost
DBEdit3	
DataSource	DataModule2.DS_T_goods_catalog
DataField	gcDescription

Тип товара	Название	Цена
Ноутбуки	Acer Aspire 1 A115-32-C2Z1 серебристый	25499
Ноутбуки	Lenovo IdeaPad 3 15IGL05 серый	27955
Ноутбуки	ASUS VivoBook M513UA-BQ080 черный	49999
Системные блоки	DEXP Mars E317	41999
Системные блоки	ASUS ROG Strix GL10CS-RU077T	64990

Описание
1366x768, TN+film, Intel Celeron N4500, 2 x 1.1 ГГц, RAM 8 ГБ, eMMC 128 ГБ, Intel UHD Graphics

Вставить Изменить Удалить

Тип товара: Ноутбуки
Название: Acer Aspire 1 A115-32-C2Z1 серебристый
Цена: 25499

Описание: 1366x768, TN+film, Intel Celeron N4500, 2 x 1.1 ГГц, RAM 8 ГБ, eMMC 128 ГБ, Intel UHD Graphics

Запомнить Отменить

Рис. 2.30 – Окно формы редактирования товаров

Теперь необходимо прописать код для обработки события нажатия на кнопки. За это действие отвечает обработчик **OnClick**. Для его определения необходимо выполнить двойное нажатие левой кнопки мыши на компоненте **TButton** или в области события **OnClick** в окне инспектора объектов. Шаблон процедуры обработчика генерируется средой разработки автоматически, поэтому необходимо будет прописывать только код тела процедуры (между **begin** и **end**).

Компонент **TADOTable** позволяет выполнять добавление информации в таблицу при помощи двух методов **Insert** и **Append**. Оба метода переводят набор данных **TADOTable** (в нашем случае **T_goods_catalog**) в состояние добавления записи **dsInsert**. Их различие заключается в том, что метод **Append** добавляет запись в конец набора данных, в то время как метод **Insert** добавляет ее после текущей записи. Для перехода в состояние **dsInsert** необходимо, чтобы набор данных находился в режиме просмотра **dsBrowse**.

Определим обработчик нажатия кнопки **InsertButton**, т. е. обработчик события **OnClick**:

```
procedure TForm4.InsertButtonClick(Sender: TObject);
begin
  if DataModule2.T_goods_catalog.State = dsBrowse then
  begin
    // Отключим возможность изменения курсора НД в компоненте DBGrid1
    DBGrid1.Enabled := False;
    // Сделаем невидимой панель с кнопками Вставить, Изменить,
    // Удалить
    InsertEditDeletePanel.Visible := False;
    // Сделаем видимой панель для ввода значения в запись
```

```

    PanelToInputValues.Visible := True;
    // Переведем НД в режим добавления записи, используя
    // метод Append
    DataModule2.T_goods_catalog.Append;
end;
end;

```

Аналогичным образом определим обработчик нажатия кнопки **EditButton**:

```

procedure TForm4.EditButtonClick(Sender: TObject);
begin
    if DataModule2.T_goods_catalog.State = dsBrowse then
    begin
        DBGrid1.Enabled := False;
        InsertEditDeletePanel.Visible := False;
        PanelToInputValues.Visible := True;
        // Переведем НД в режим редактирования записи, используя
        // метод Edit
        DataModule2.T_goods_catalog.Edit;
    end;
end;

```

Метод **Edit** переводит набор данных **T_goods_catalog** в состояние редактирования записи **dsEdit**. Редактирование значений полей осуществляется в компонентах **DBLookupComboBox1**, **DBEdit1**, **DBEdit2**, **DBEdit3**. При этом необходимо, чтобы набор данных находился в режиме просмотра **dsBrowse**.

Определим обработчик нажатия кнопки **DeleteButton**:

```

procedure TForm4.DeleteButtonClick(Sender: TObject);
begin
    if DataModule2.T_goods_catalog.State = dsBrowse then
        if MessageDlg('Подтвердите удаление записи', mtConfirmation,
            [mbYes, mbNo], 0) = mrYes then
            DataModule2.T_goods_catalog.Delete;
end;

```

Если набор данных **T_goods_catalog** находится в режиме просмотра **dsBrowse**, то при выполнении функции **MessageDlg** будет вызвано окно диалога. При нажатии кнопки **Yes** произойдет удаление текущей записи в наборе данных **T_goods_catalog**, при условии, что данный элемент каталога нигде не используется и на складе имеется 0 единиц товара.

Теперь зададим обработчик нажатия кнопки **PostButton**:

```

procedure TForm4.PostButtonClick(Sender: TObject);
begin
    if DataModule2.T_goods_catalog.State in [dsInsert, dsEdit] then
    begin
        // Выполним операцию сохранения
        DataModule2.T_goods_catalog.Post;
    end;
end;

```



```

// Сделаем видимой панель с кнопками Вставить, Изменить,
// Удалить
InsertEditDeletePanel.Visible := True;
// Сделаем невидимой панель для ввода значения в запись
PanelToInputValues.Visible := False;
// Включим возможность изменения курсора НД
// в компоненте DBGrid1
DBGrid1.Enabled := True;
end;
end;

```

Если набор данных находится в режиме добавления новой записи или редактирования, то происходит выполнение метода **Post**, который запоминает текущее состояние записи в таблице. После запоминания набор данных переводится в режим просмотра **dsBrowse**.

Определим обработчик нажатия кнопки **CancelButton**:

```

procedure TForm4.CancelButtonClick(Sender: TObject);
begin
  if DataModule2.T_goods_catalog.State in [dsInsert, dsEdit] then
  begin
    // Выполним операцию отмены
    DataModule2.T_goods_catalog.Cancel;
    InsertEditDeletePanel.Visible := True;
    PanelToInputValues.Visible := False;
    DBGrid1.Enabled := True;
  end;
end;

```

В этом случае нахождение набора данных в режиме добавления новой записи или в режиме редактирования приводит к выполнению метода **Cancel**. Этот метод отменяет запоминание записи в таблице и переводит набор данных в режим просмотра **dsBrowse**.

Для запрета перевода набора данных в состояние добавления и изменения данных, а также для удаления записей непосредственно из компонента **DBGrid1**, установим для него свойство **ReadOnly** в значение **True**.

При просмотре в компоненте **DBGrid1** по умолчанию выделяется ячейка, чтобы ее можно было редактировать, поэтому при отключении режима редактирования было бы удобно, чтобы выделенная строка подсвечивалась целиком. Для достижения этого визуального эффекта через инспектор объектов у **DBGrid1** в списке параметров **Options** необходимо установить свойство **dgRowSelect** в значение **true**.

При переходе по записям в наборах данных (**TADOTable**, **TADOQuery** и т. п.), возникают события **BeforeScroll** (перед переходом на другую запись) и **AfterScroll** (после перехода).

В нашем случае необходимо, чтобы после перемещения по записям в наборах данных компонентов **TADOQuery** в наборе данных **TADOTable** связанным с редактируемой таблицей выбиралась соответствующая за-

пись. Для этого необходимо настроить событие **AfterScroll** у компонента **Q_v_goods_catalog**:

```
procedure TDataModule2.Q_v_goods_catalogAfterScroll(DataSet:
  TDataSet);
begin
  // проверка количества отображаемых записей
  if Q_v_goods_catalog.RecordCount > 0 then
    // выполнение синхронизации методом Locate
    T_goods_catalog.Locate('IDgc', Q_v_goods_catalog.IDgc.Value, [])
end;
```

Для того чтобы данный фрагмент кода заработал необходимо чтобы для компонента **Q_v_goods_catalog** были определены все поля.

При срабатывании события **AfterScroll** проверяется, имеются ли отображаемые записи (количество записей больше нуля), после чего в **T_goods_catalog** будет происходить поиск записи по значению **Q_v_goods_catalog.IDgc**. Переход на нужную запись осуществляется при помощи метода **Locate**.

Метод **Locate** ищет первую запись, удовлетворяющую критерию поиска, и если такая запись найдена, то делает ее текущей. В этом случае в качестве результата возвращается значение **True**. Если поиск был неуспешен, то возвращается значение **False**:

```
function Locate(const KeyFields: String; const KeyValues: Variant;
  Options: TLocateOptions): Boolean;
```

Приведем параметры метода **Locate**:

- список **KeyFields** определяет поле или несколько полей, по которым ведется поиск, в виде строкового выражения. В случае нескольких поисковых полей их названия разделяются точкой с запятой;
- критерии поиска задаются в вариантном массиве **KeyValues** так, что *i*-е значение в параметре **KeyValues** ставится в соответствие *i*-му полю в параметре **KeyFields**. В случае поиска по одному полю в массиве **KeyValues** указывается одно значение;
- параметр **Options** позволяет указать необязательные значения режимов поиска: **loCaseInsensitive** – поиск ведется без учета регистра букв, **oPartialKey** – запись считается удовлетворяющей условию поиска, если она содержит часть поискового запроса. В случае работы с числовыми полями можно не указывать опций, а оставить пустые квадратные скобки [].

Компонент **TADOQuery**, имеет следующую особенность: после выполнения запроса на чтение к базе данных происходит отображение записи НДС в неизменном виде независимо от того, изменялось ли после запроса содержимое таблицы БД. После обновления информации в базе данных приходится закрывать и снова открывать компонент **TADOQuery**, т. е. повторно выполнять запрос к базе данных. В связи с этим нам необходимо добавить обработчики событий **AfterPost** и **AfterDelete** для компонента **TADOTable**. Обновление набора данных в **TADOQuery** возможно путем

его закрытия и открытия, которое осуществляется при помощи методов **Close()** и **Open()** или же изменения свойства **Active** сначала в значение **False**, а потом в значение **True**. Также обновить данные можно при помощи метода **Requery()**.

Поскольку мы сделали редактирование каталога товара, то для компонента **T_goods_catalog** пропишите следующие события:

```
procedure TDataModule2.T_goods_catalogAfterDelete(DataSet: TDataSet);
begin
    // обновление данных в НД Q_v_goods_catalog
    Q_v_goods_catalog.Requery();
    // обновление данных в НД Q_v_storage
    Q_v_storage.Requery();
end;

procedure TDataModule2.T_goods_catalogAfterPost(DataSet: TDataSet);
var pos:TBookmark;
begin
    // создание для текущей записи объекта-закладки
    pos := Q_v_goods_catalog.GetBookmark;
    Q_v_goods_catalog.Close();
    Q_v_goods_catalog.Open();
    // перемещение курсора БД на запись, определяемую закладкой
    Q_v_goods_catalog.GotoBookmark(pos);
    Q_v_storage.Requery();
end;
```

Для того чтобы позиционирование курсора в наборе данных сохранялось в момент редактирования используется закладка (тип данных **TBookmark**). Тип **TBookmark** является указателем на экземпляр типа «закладка». В НД можно использовать закладки аналогично тому как закладки используются в жизни. При помощи закладок легко пометить место в НД и потом вернуться к нему.

При выполнении действий с НД, имеющим большое количество записей, что влечет за собой частое изменение местоположения курсора БД, в визуальном компоненте, показывающем записи (например, **TDBGrid**) или текущую запись (например, **TDBEdit**), будет возникать эффект прокрутки записей, который не всегда может устраивать пользователя. Кроме того, при смене местоположения курсора БД, т. е. при смене текущей записи НД, необходимо дополнительное время для отражения в визуальном компоненте произошедших изменений.

Для устранения эффекта прокрутки используются два метода:

```
procedure DisableControls;
procedure EnableControls;
```

Первый метод отключает связь с визуальным компонентом, а второй восстанавливает ее. При этом в таблице компонента **TDBGrid** не будет видно эффекта прокрутки записей. Наоборот, у пользователя возникнет

иллюзия, что курсор БД сразу переустановился с прежней записи набора данных на нужную запись.

Так, например, можно изменить событие **AfterPost** или **AfterDelete**, например:

```
procedure TDataModule2.T_goods_catalogAfterDelete(DataSet: TDataSet);
begin
    Q_v_goods_catalog.DisableControls();
    Q_v_goods_catalog.Requery();
    Q_v_storage.Requery();
    Q_v_goods_catalog.EnableControls();
end;
```

Такой режим обновления отображения позволит избежать мерцания записей в **TDBGrid** при обновлении отображения данных.

КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Что из себя представляет *ODBC*?
2. Чем контейнер *DataModule* отличается от *Form*?
3. Что содержит *ConnectionString*?
4. Что из себя представляет порт (для *MySQL* - 3306)?
5. Дайте расшифровку аббревиатуры *SQL*.
6. Для чего применяется компонент *TADOQuery*?
7. Чем *Events* отличаются от *Properties*?
8. Опишите назначение компонента *TDataSource*.
9. Дайте характеристику обычным и модальным формам.
10. Чем свойство *Caption* отличается от *Name*?
11. Для чего применяется команда *Use Unit*?
12. За что отвечает свойство компонентов – *Align*?
13. Охарактеризуйте свойство *Margins*?
14. В каких режимах могут находиться наборы данных?
15. Перечислите способы обновления НД *TADOQuery*.
16. В какой момент срабатывает событие *AfterScroll*?
17. Для чего применяется метод *Locate*?
18. Что из себя представляет *TBookmark*?
19. Каким образом можно устранить эффект прокрутки?